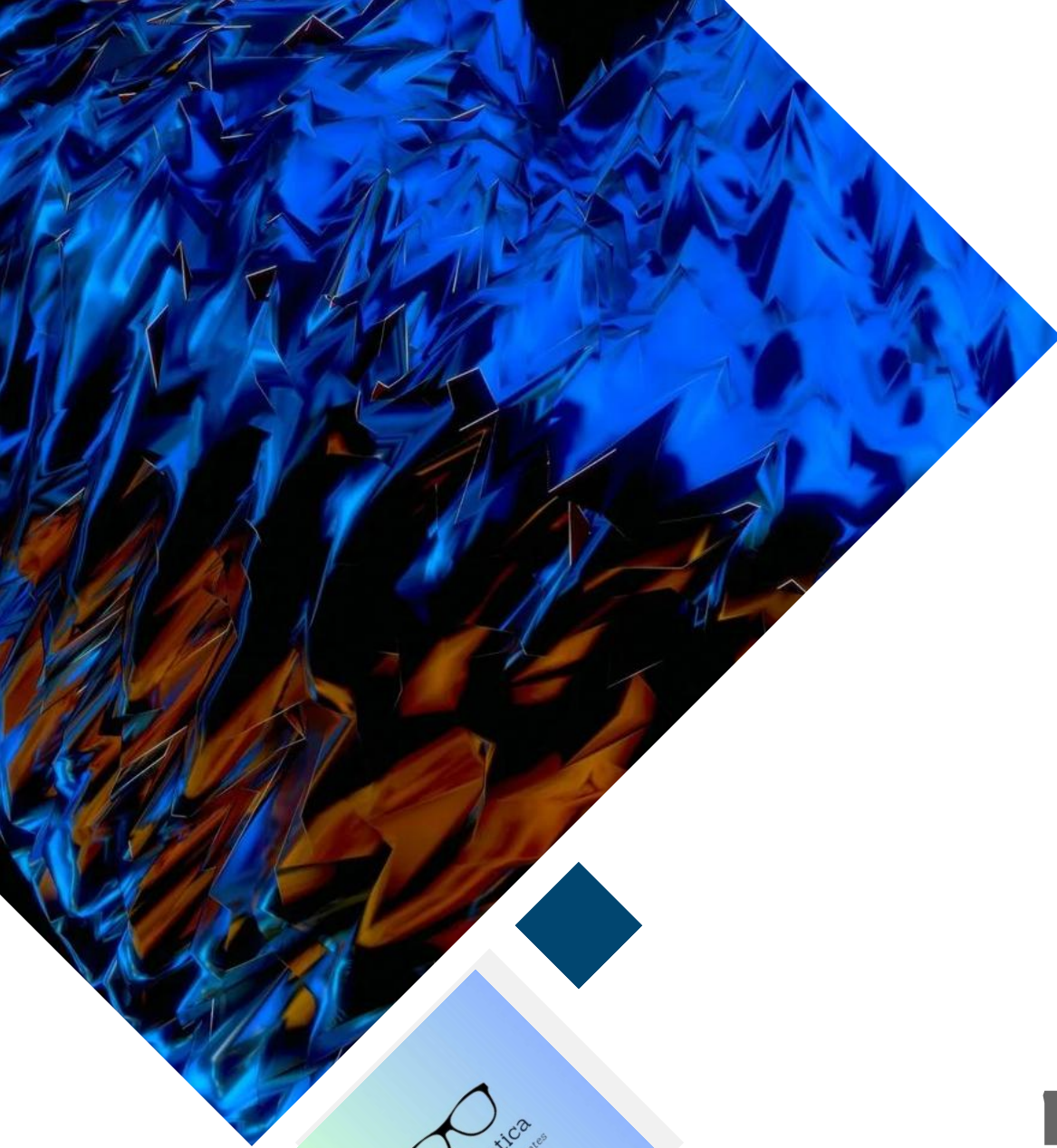




Manual Técnico de SIMO

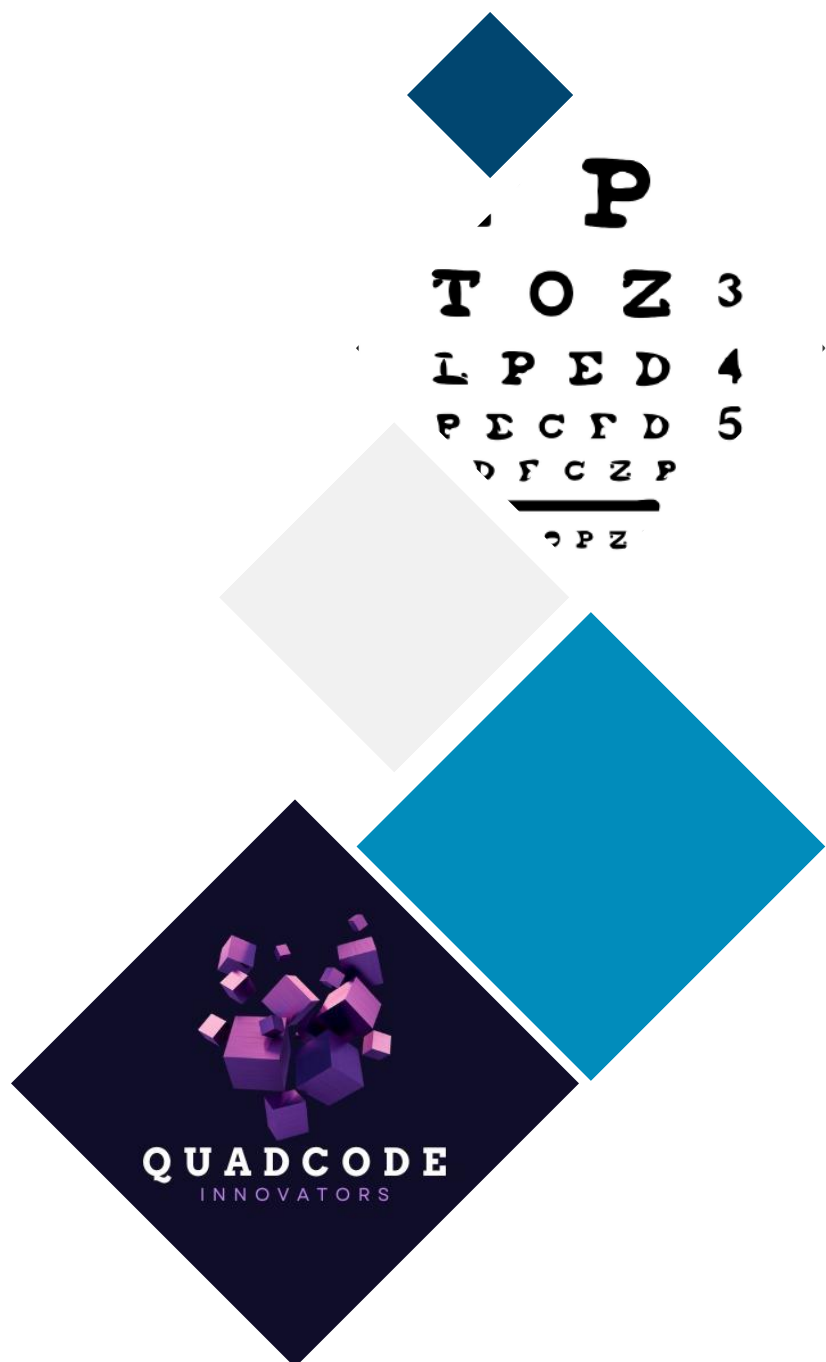
QUADCODE INNOVATORS.



**Sistema
Integral Más
Óptica**

Contenido

- 03** Introducción
- 04** Objetivos y Alcance
- 07** Funcionalidades Claves
- 10** Tecnologías Utilizadas
- 11** Requerimientos tecnicos
- 12** Instalación
- 14** Configuración
- 15** Desarrollo Del Codigo Fuente
- 18** Estructura Del Codigo Fuente
- 19** Privilegios de Usuario
- 20** Base De Datos
- 24** Diagrama E-R
- 25** Configuración y Ejecución
- 26** Resumen



Contenido De Figuras

18 Figura 1 - Carpeta de codigo.

24 Figura 2 - Diagrama E-R.

Introducción



El propósito de este manual técnico es proporcionar una guía completa para la implementación, configuración y uso del Sistema Integral Más Óptica (SIMO). Está dirigido a personal técnico y operativo para garantizar una correcta instalación y administración del sistema.

A continuación, se presentan detalles sobre las tecnologías y herramientas utilizadas en el entorno de desarrollo del SIMO:

JavaFX: JavaFX es una plataforma para el desarrollo de aplicaciones de escritorio ricas en gráficos y multimedia en Java. Ofrece una arquitectura moderna y un conjunto de APIs que facilitan la creación de interfaces de usuario interactivas y visualmente atractivas. En SIMO, JavaFX se utiliza para construir la interfaz gráfica de usuario (GUI), proporcionando una experiencia de usuario fluida y eficiente.

JavaFX Scene Builder: JavaFX Scene Builder es una herramienta gráfica que permite diseñar interfaces de usuario para aplicaciones JavaFX de manera visual y sin necesidad de escribir código FXML a mano. Permite arrastrar y soltar elementos de la interfaz, configurar sus propiedades y generar el archivo FXML que describe la interfaz. En SIMO, Scene Builder se usa para crear y ajustar las pantallas y controles de la aplicación de manera intuitiva.

IntelliJ IDEA: IntelliJ IDEA es un entorno de desarrollo integrado (IDE) para Java y otros lenguajes de programación. Proporciona herramientas avanzadas para el desarrollo, depuración y prueba de aplicaciones. En el contexto de SIMO, IntelliJ IDEA se utiliza para escribir, gestionar y depurar el código fuente del proyecto, facilitando un flujo de trabajo productivo y eficiente.

Maven: Maven es una herramienta de gestión y construcción de proyectos para Java. Permite gestionar las dependencias del proyecto, automatizar el proceso de construcción y generar informes. En SIMO, Maven se emplea para administrar las librerías necesarias, construir el proyecto y empaquetar la aplicación en un archivo JAR ejecutable.

Objetivos



El objetivo general del SIMO es optimizar las operaciones comerciales de Más Óptica Especialistas en Lentes, mejorando la eficiencia en la gestión de ventas, inventario, y atención al cliente.

1. Automatizar el proceso de ventas: Facilitar la creación, seguimiento y gestión de pedidos y pagos. 
2. Optimizar el control de inventario: Mantener actualizadas las existencias y generar alertas de bajo stock. 
3. Mejorar la gestión de cliente: Clasificar clientes y automatizar notificaciones relevantes. 
4. Proveer reportes detallados: Ofrecer informes personalizables sobre ventas, inventario y clientes. 

Alcance

SIMO es una aplicación de software que integra la gestión de ventas, inventario y clientes en una plataforma unificada. Está diseñado para mejorar la eficiencia operativa y la experiencia del cliente en Más Óptica Especialistas en Lentes.

**1**

Usuarios del Sistema

El sistema está destinado a:

- Personal de ventas
- Personal de inventario
- Administradores del sistema
- Personal de atención al cliente

2

Limitaciones y Exclusiones

- El sistema no gestiona aspectos financieros como contabilidad completa o informes fiscales.
- No incluye funcionalidades para la gestión de recursos humanos o nómina.

Gestión de Ventas



Creación y Seguimiento de Pedidos

Permite registrar nuevos pedidos, hacer seguimiento y actualizar su estado.



Registro de Pagos y Abonos

Facilita el registro de pagos realizados y abonos aplicados a las cuentas de clientes.

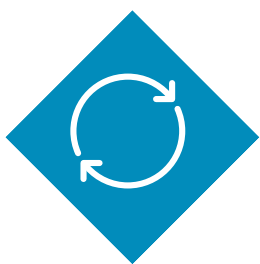


Historial de Ventas Detallado:

- Proporciona un registro completo de todas las transacciones realizadas.

Gestión de Inventario

Permite supervisar y controlar el stock de productos. Implica la planificación, el seguimiento y la optimización de los niveles de inventario



Actualización Automática de Existencias:

Las existencias se actualizan en tiempo real con cada venta y recepción de productos.



Alertas de Bajo Stock:

Notifica al personal cuando los niveles de inventario están por debajo del umbral definido.



Generación de Informes de Inventario

Permite crear informes detallados sobre el estado del inventario.

Gestión de Clientes



Categorización de Clientes

Clasifica a los clientes en categorías como "bueno", "malo" o "regular" según su historial.

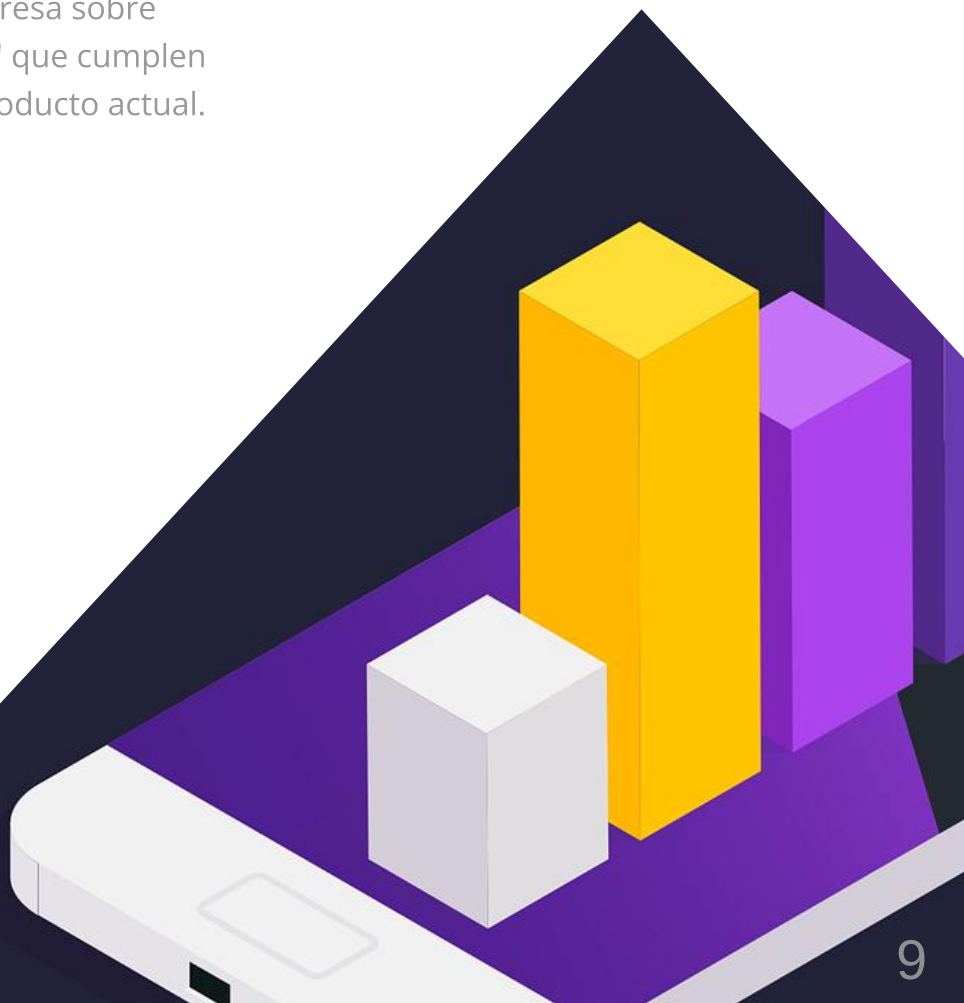


Notificaciones Automáticas a Más Óptica

Informa a la empresa sobre clientes "buenos" que cumplen un año con su producto actual.

Otras Características

- **Interfaz de Usuario Intuitiva:** Facilita la navegación y uso del sistema con una interfaz amigable.
- **Reportes Personalizables:** Permite generar reportes adaptados a las necesidades específicas del usuario.
- **Integración con Sistemas Existentes:** Compatible con otros sistemas que la empresa pueda estar utilizando.



Tecnologías Utilizadas

Base de datos

MariaDB: Sistema de gestión de base de datos relacional utilizado para almacenar datos.
Manejador de Base de Datos

Manejador de Base de Datos

SQLyog Community: Herramienta para diseño y administración de bases de datos MariaDB.
Lenguaje de Programación

Lenguaje de Programación

Java: Lenguaje utilizado para el desarrollo del software.

Control de Versiones

Git: Sistema de control de versiones para gestionar el código fuente del proyecto.

Framework

JavaFX: Framework utilizado para construir interfaces gráficas y gestionar eventos de usuario.

Diseño de Interfaz

JavaFX Scene Builder: Herramienta para diseñar interfaces gráficas de usuario en JavaFX.
Framework

Gestor de Proyectos

Maven: Herramienta para la gestión de dependencias y el proceso de construcción del proyecto.

IDE

IntelliJ IDEA: Entorno de desarrollo integrado para programar en Java.

Requerimientos Técnicos

Especificaciones del equipo, programas y aplicaciones necesarios, así como sus versiones y compatibilidades.



Requerimientos Mínimos de Hardware

- Procesador: Intel Core i5 o equivalente.
- Memoria RAM: 8 GB.
- Espacio en Disco: 1 GB para instalación y espacio adicional para datos.
- Resolución de Pantalla: 1366 x 768 píxeles.



Requerimientos Mínimos de Software

- Sistema Operativo: Windows 10
- Java Development Kit (JDK): Versión 19 o superior.
- MariaDB: Versión 10.5 o superior.
- Maven: Versión 3.8 o superior.



Herramientas Utilizadas para el Desarrollo

- JavaFX Scene Builder: Para el diseño visual de la interfaz.
- IntelliJ IDEA: Para el desarrollo y depuración del código.
- Git: Para el control de versiones.
- SQLyog Comunnity: Para el diseño y gestión de la base de datos.

Instalación: Requisitos



Java Development Kit (JDK)

Asegúrese de tener instalado JDK 19 o superior



MariaDB

Debe estar instalado y en ejecución en el sistema.



Maven

Asegúrese de tener Maven instalado para la gestión del proyecto.

Instalación:

Pasos para la descarga

- I. **Clonar el Repositorio:** Clone el repositorio del proyecto desde GitHub utilizando el comando ``git clone``
`https://github.com/xGeorgeZ/SIMO`
- II. **Configurar la Base de Datos:** Edite el archivo ``DatabaseConnection.java`` para configurar los parámetros de conexión a la base de datos MariaDB.
- III. **El Proyecto:** Ejecute ``mvn clean install`` para compilar y empaquetar el proyecto.
- IV. **Ejecutar la Aplicación:** Ejecute el archivo ``simo.jar`` generado en la carpeta ``target/`` utilizando ``java -jar target/simo.jar``.

Configuración



Configuración Inicial

- Base de Datos: Asegúrese de que la base de datos esté configurada según el esquema definido en `simo.sql`.
- Parámetros del Sistema: Ajuste los parámetros del sistema en los archivos de configuración según las necesidades de su entorno.



Configuración de Conexión a la Base de Datos

- Archivo `DatabaseConnection.java`: Configure el archivo con los detalles de la base de datos, como la URL, el nombre de usuario y la contraseña.

Desarrollo del Código Fuente

```
package com.quadcode.simo;

import com.quadcode.simo.controller.LoginController;
import com.quadcode.simo.dao.LoginDao;
import com.quadcode.simo.service.LoginService;
import javafx.application.Application;
import javafx.application.Platform;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.stage.Stage;
import javafx.stage.StageStyle;

/**
 * Main.java
 * -----
 * Clase principal que inicia la aplicacion JavaFX.
 * Esta clase configura y muestra la ventana de inicio de sesión.
 * -----
 * @author: Avila Carrillo Jorge Armando
 * Fecha de creacion: 19 - MAYO - 2024
 * Ultima Actualizacion: 25 - JULIO - 2024
 * -----
 */

public class Main extends Application {

    // Atributos que capturan la posicion del cursor
    private double xMouse, yMouse;

    // Punto de entrada para la aplicacion.
    public static void main(String[] args) {
```

Desarrollo del Código Fuente

```
launch(args);
}
/**
 * Inicializa la aplicacion y carga la interfaz de usuario del Login.
 *
 * @param primaryStage El escenario principal de la aplicación.
 * @throws Exception Si ocurre algun error al cargar el archivo FXML.
 */
@Override
public void start(Stage primaryStage) throws Exception {
    // Imagen para el logo de la aplicacion
    Image logo = new Image("/com/quadcode/simo/img/LogoMasOptica.jpg");

    // Carga el archivo FXML para la vista de inicio de sesión
    FXMLLoader loader = new
FXMLLoader(getClass().getResource("/com/quadcode/simo/view/LoginView.fxml"));

    // Carga el objeto raiz desde el archivo FXML.
    Parent root = loader.load();

    // Obtiene la hoja de estilos
    root.getStylesheets().add(getClass().getResource("/com/quadcode/simo/styles/styles.cs
s").toExternalForm());

    // Obtiene el controlador de vista del Login
    LoginController loginController = loader.getController();

    // Configura el servicio de usuario y el DAO
    LoginDao loginDao = new LoginDao();
    LoginService loginService = new LoginService(loginDao);

    // Establece el servicio de usuario en el controlador
    loginController.setLoginService(loginService);
```


Desarrollo del Código Fuente

```
// Configura la escena principal con el objeto raiz cargado desde FXML
Scene scene = new Scene(root);
primaryStage.setScene(scene);
primaryStage.initStyle(StageStyle.UNDECORATED);
primaryStage.setTitle("SIMO");
primaryStage.show();
primaryStage.getIcons().add(logo);

// Obtiene la posicion del cursor, y mueve la ventana mientras esta es presionada por
// el usuario
// Esto es por que la ventana esta tipo UNDECORATED
root.setOnMousePressed(mouseEvent -> {
    xMouse = mouseEvent.getSceneX();
    yMouse = mouseEvent.getSceneY();
});
root.setOnMouseDragged(mouseEvent -> {
    primaryStage.setX(mouseEvent.getScreenX() - xMouse);
    primaryStage.setY(mouseEvent.getScreenY() - yMouse);
});
}
```

ESTRUCTURA DEL CÓDIGO FUENTE

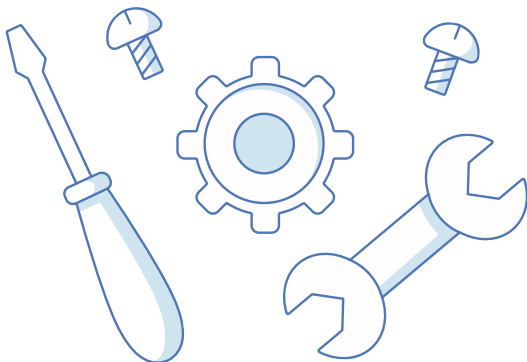


ESTRUCTURA

- ``src/main/java/com/quadcode/simo/``: Código fuente principal del proyecto.
- ``controller/``: Clases que gestionan la lógica de la interfaz de usuario.
- ``dao/``: Clases para el acceso a datos.
- ``model/``: Clases que representan los datos del negocio.
- ``service/``: Clases que contienen la lógica de negocio.

DIRECTRICES DE CODIFICACIÓN

1. **Estilo de Código:** Sigue las convenciones de codificación de Java.
2. **Documentación:** Usa comentarios claros y concisos para describir la funcionalidad del código.



CARPETAS DE CÓDIGO INCLUYENDO (CONTROLADORES, DAO Y MODEL)

```

└─ java
  └─ com.quadcode.simo
    └─ controller
      └─ AbonosController
      └─ ClienteController
      └─ ConfiguracionController
      └─ HomeController
      └─ InventarioController
      └─ LoginController
      └─ NavBarController
      └─ NuevaVentaController
      └─ PacienteController
      └─ VentasController
    └─ dao
      └─ ClienteDao
      └─ DatabaseConnection
      └─ LoginDao
    └─ model
      └─ Cliente
      └─ ClienteDetalle
      └─ Direccion
      └─ Paciente
      └─ Referencia
      └─ UserLogin
  
```

Figura 1 - Carpeta de código.

Privilegios de Usuario



Administrador

Acceso completo a todas las funcionalidades del sistema.



Vendedor

Acceso a la gestión de ventas y clientes.



Gestor de Inventario

- Acceso a la gestión de inventario y reportes.

Base De Datos: Tablas

Descripción General

Base de Datos: MariaDB se utiliza para almacenar toda la información relevante del sistema, como ventas, productos, y clientes.

Esquema de la Base de Datos

Tablas Principales:

Clientes: Almacena información sobre los clientes.

Productos: Almacena información sobre los productos disponibles.

Ventas: Registra las transacciones de ventas realizadas.

Tablas Secundarias:

Abonos: Registra los abonos realizados en una venta, incluyendo la fecha y el monto.

Detallesventa: Detalla los productos y planes involucrados en cada venta, incluyendo el cliente, método de pago y características específicas del producto.

Diagnostico: Guarda información sobre el diagnóstico del paciente, incluyendo el tipo de material y tratamiento del lente, así como medidas específicas para cada ojo.

Direcciones: Almacena las direcciones de clientes, referidos y otros contactos, incluyendo detalles como calle, código postal y referencias.

Micas: Registra información sobre los tipos de micas disponibles, incluyendo precios para contado y crédito.

Base De Datos: Tablas

Descripción General

Ojoderecho: Almacena las medidas ópticas específicas para el ojo derecho del paciente, como esfera, cilindro y eje.

Ojoizquierdo: Similar a la tabla ojoderecho, pero para el ojo izquierdo del paciente.

obtener_clientes_detalle: Almacena detalles completos sobre clientes y sus referidos, incluyendo información de contacto y direcciones tanto del cliente como del referido.

Pacientes: Contiene información sobre los pacientes, vinculándolos a un cliente específico.

pacientes_detalle: Guarda información detallada sobre los diagnósticos de los pacientes, incluyendo datos del cliente asociado, especificaciones del tratamiento óptico, y medidas de lentes para ambos ojos.

Referidos: Contiene información sobre las personas que refieren nuevos clientes, incluyendo su nombre, teléfono y relación con el cliente.

Usuarios: Almacena los datos de los usuarios del sistema, incluyendo nombre de usuario y contraseña.

Base De Datos: Objetos

Funciones

obtener_id_cliente_por_nombre: Esta función recibe un nombre de cliente como parámetro y devuelve el Id del cliente correspondiente. Si hay varios clientes con el mismo nombre, solo se devuelve el primer resultado encontrado.

obtener_nombre_clientes: Esta función recibe un patrón de búsqueda como parámetro y devuelve una cadena de texto con los nombres de los clientes que coinciden con ese patrón. Los nombres se concatenan en una sola cadena, separados por comas. Se limita a los primeros 5 resultados ordenados alfabéticamente.

Procedimientos Almacenados:

InsertarClienteConReferido: Inserta un nuevo cliente y su referido asociado en la base de datos. Incluye la dirección del cliente y del referido.

insertar_diagnostico: Inserta un diagnóstico para un paciente asociado a un cliente. Incluye datos sobre lentes y el tratamiento.

modificarCliente: Actualiza la información de un cliente y su referido asociado, incluyendo direcciones.

modificarPaciente: Actualiza la información del paciente y su diagnóstico asociado. Si el diagnóstico ya existe, actualiza los datos; si no, inserta un nuevo registro.

actualizar_venta: Actualiza la información de una venta existente, como la fecha de la venta, el total de la venta y el saldo actual.

eliminar_abono: Elimina un abono específico registrado en la base de datos, identificado por su ID.

insertar_abono: Inserta un nuevo abono en la base de datos, asociándolo con una venta existente y especificando la fecha y el monto del abono.

insertar_micas: Agrega un nuevo tipo de mica a la base de datos, incluyendo su tipo y precios tanto al contado como a crédito.

insertar_productos: Introduce un nuevo producto en la base de datos, especificando su nombre, stock, precios y descripción.

insertar_ventas: Registra una nueva venta, vinculando la venta con un paciente, un plan de pagos y actualizando el total y saldo actual.

Base De Datos: Objetos

Procedimientos Almacenados

modificar_abono: Modifica los detalles de un abono existente, como la fecha y el monto, para reflejar cambios necesarios.

modificar_mica: Actualiza la información de una mica existente, como su tipo y precios, para reflejar cambios en los costos o características.

modificar_productos: Actualiza los detalles de un producto en la base de datos, como su nombre, stock, precios y descripción, para mantener la información actualizada.

Vistas

obtener_clientes_detalle: Proporciona una vista consolidada de la información de los clientes, incluyendo datos de contacto, detalles del referido asociado y direcciones tanto del cliente como del referido.

pacientes_detalle: Ofrece una vista detallada de los diagnósticos de pacientes, con información sobre el paciente, el cliente asociado, y las especificaciones del tratamiento óptico y medidas de lentes para ambos ojos.

obtener_venta_detalle: Actúa como una plantilla o marcador de posición. La vista no muestra datos reales sino valores predeterminados para los campos como Id, NombreCliente, NombreProducto, etc.

obtener_micas: Actúa como una plantilla o marcador de posición. La vista no muestra datos reales sino valores predeterminados para los campos como Id, Tipo, Contado y Crédito.

obtener_productos: Actúa como una plantilla o marcador de posición. La vista no muestra datos reales sino valores predeterminados para los campos como Id, Nombre, Stock, PrecioContado, PrecioCredito y Descripcion.

Diagrama E-R

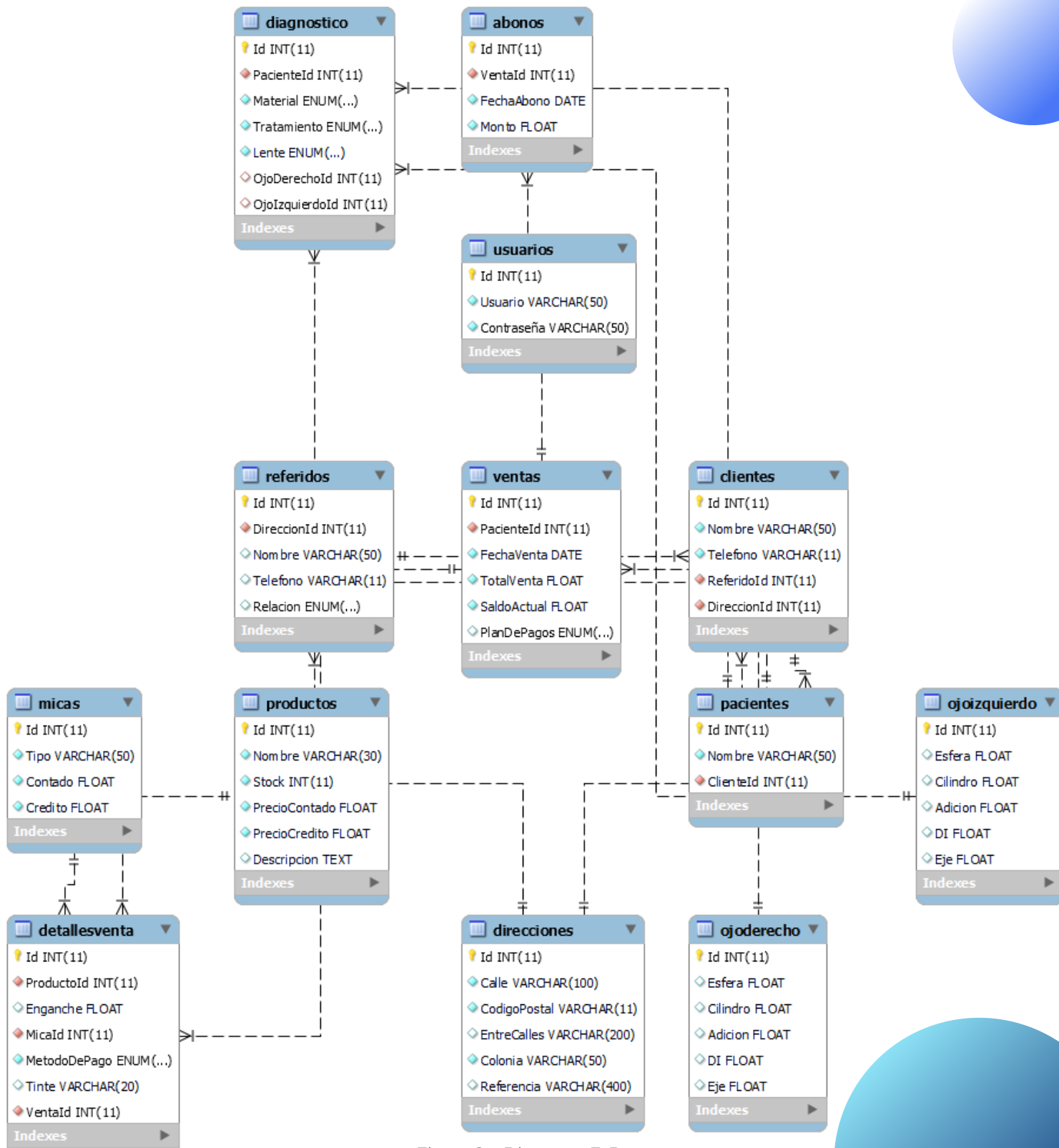


Figura 2 - Diagrama E-R.

Configuración y Ejecución

Ajustar y establecer los parámetros necesarios para que el software funcione según los requisitos.



Requisitos

- Java JDK: Debe estar instalado y configurado en el sistema.
- MariaDB: Debe estar en funcionamiento y configurado con el esquema de base de datos proporcionado.



Configuración

- Base de Datos: Configure la conexión a la base de datos en el archivo ``DatabaseConnection.java``.
- Parámetros del Sistema: Ajuste los parámetros del sistema en el archivo de configuración.



Ejecución

Iniciar la Aplicación: Ejecute el archivo ``simo.jar`` utilizando el comando ``java -jar target/simo.jar``.

Resumen

El Sistema Integral Más Óptica (SIMO) está diseñado para mejorar la eficiencia en la gestión de ventas, inventario y clientes para Más Óptica Especialistas en Lentes. Este manual proporciona toda la información necesaria para la instalación, configuración y uso del sistema.

Nota Adicional

Para cualquier actualización o modificación en el sistema, consulte el repositorio del proyecto y los documentos de especificaciones adicionales.

